



ingressaê.

pensado e desenvolvido por:

João Marcelo de Stefani Lessa – Júlia Mota Spricigo – Laura Savi Bressan
– Marcos Paulo Bittencourt Medeiros – Mateus Dantas Wendhausen

O que nos torna diferentes

- Fila Preferencial com critério justo;
- Fila Normal com ordem garantida;
- Posição em tempo real;
- Jogo enquanto aguarda;
- Fã Clubes integrados;
- Tempo limitado para compra;
- Limites por tipo de fila.

da **aula**
para a
prática

▼ estruturas

TS busca-sequencial.ts

TS fila-normal.ts

TS fila-preferencial.ts

TS insertion-sort.ts

TS pilha.ts

TS quick-sort.ts

Fila Normal

← Voltar aos shows

Humor & Hits

Arena BRB, Brasília · 04 de ago. de 2026

Normal

SUA POSIÇÃO

#3,251

⌚ ~33 minutos

Progresso

0%

⚠️ Você pode comprar apenas 2 ingressos nesta fila.

Ordem de chegada. Sem critérios adicionais. Primeiro a entrar, primeiro a ser atendido.

→ Sair da Fila

Fácil

Médio

Difícil

Hardcore

Movimentos: 0

Reiniciar



ts fila-normal.ts X

ingressae > src > app > shared > estruturas > ts fila-normal.ts > UsuarioFilaNormal

```
1  export interface UsuarioFilaNormal {
2    usuarioId: string;
3    entradaEm: Date;
4  }
5
6  export class FilaNormal {
7    private fila: UsuarioFilaNormal[] = [];
8
9    entrar(usuario: UsuarioFilaNormal): void {
10     | this.fila.push(usuario);
11   }
12
13   posicao(usuarioId: string): number {
14     | return this.fila.findIndex(u => u.usuarioId === usuarioId) + 1;
15   }
16
17   tamanho(): number {
18     | return this.fila.length;
19   }
20
21   proximo(): UsuarioFilaNormal | null {
22     | return this.fila[0] ?? null;
23   }
24
25   remover(): void {
26     | this.fila.shift();
27   }
28 }
```

shared > estruturas > fila-normal.ts
service > fila.ts

Fila Preferencial

← Voltar aos shows

Humor & Hits

Arena BRB, Brasília · 04 de ago. de 2026

👑 Preferencial

SUA POSIÇÃO

#2

⌚ ~1 minutos

Progresso

50%

⚠️ Você pode comprar apenas 1 ingresso nesta fila.

Posição baseada no seu tempo de plataforma. Usuários mais antigos ficam à frente.

→ Sair da Fila

Fácil

Médio

Difícil

Hardcore

Movimentos: 0

Reiniciar



TS fila-preferencial.ts X

ingressae > src > app > shared > estruturas > TS fila-preferencial.ts > UsuarioFila

```
1 export interface UsuarioFila {
2   usuarioId: string;
3   anosNaPlataforma: number;
4   entradaEm: Date;
5 }
6
7 export class FilaPreferencial {
8   private fila: UsuarioFila[] = [];
9
10  entrar(usuario: UsuarioFila): void {
11    let i = 0;
12
13    // Quem tem mais anos entra na frente
14    // Se tiver o mesmo tempo, respeita a ordem de chegada
15    while (
16      i < this.fila.length &&
17      this.fila[i].anosNaPlataforma >= usuario.anosNaPlataforma
18    ) {
19      i++;
20    }
21
22    this.fila.splice(i, 0, usuario);
23  }
```

```
24
25  posicao(usuarioId: string): number {
26    return this.fila.findIndex(u => u.usuarioId === usuarioId) + 1;
27  }
28
29  tamanho(): number {
30    return this.fila.length;
31  }
32
33  proximo(): UsuarioFila | null {
34    return this.fila[0] ?? null;
35  }
36
37  remover(): void {
38    this.fila.shift();
39  }
40 }
```

shared > estruturas > fila-preferencial.ts
service > fila.ts

Insertion Sort

Histórico de Ingressos

 Próximos Eventos

 Histórico

Show Imagine Dragons — Loom Tour

 19 Nov 2025

Normal

Confirmado

Show Taylor Swift — Eras Tour

 09 Mar 2026

Normal

Confirmado

Show Coldplay — Music Of The Spheres

 21 Apr 2026

Preferencial

Confirmado

Show The Weeknd — After Hours

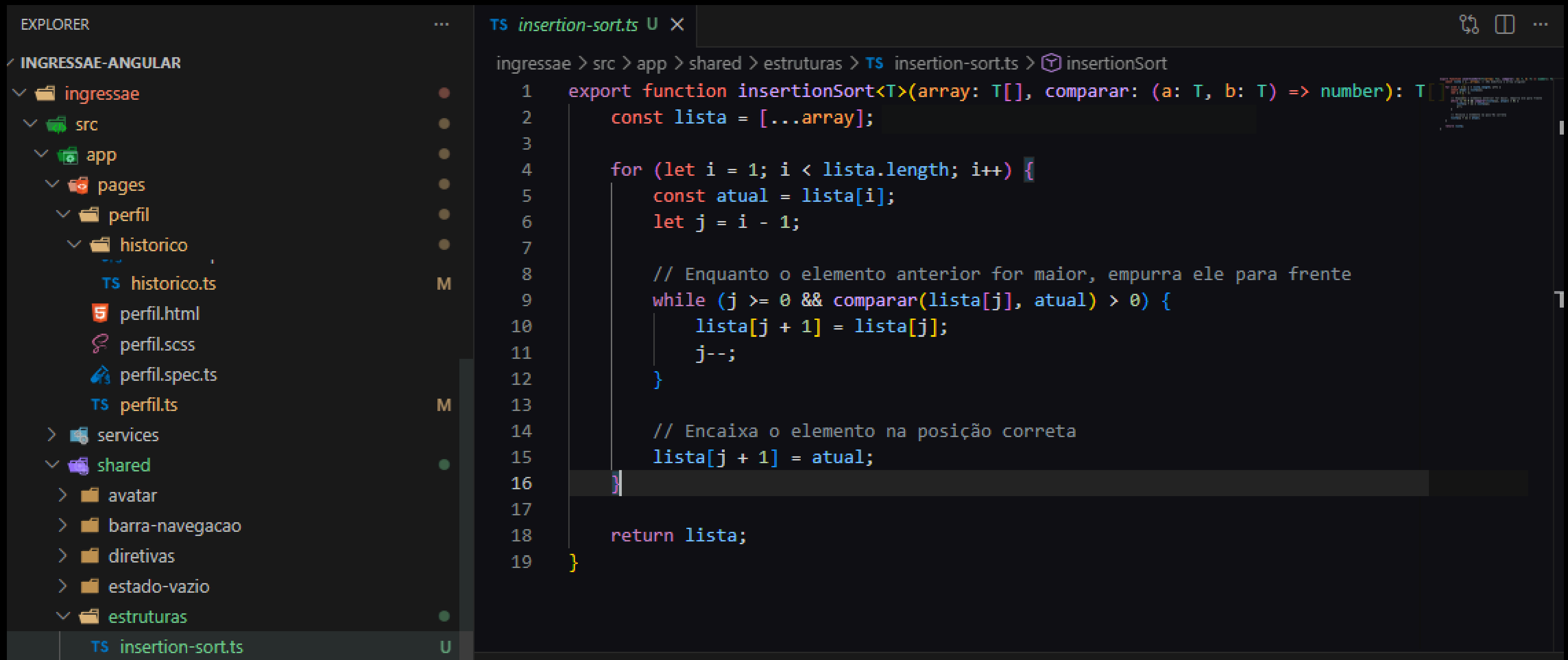
Preferencial

Confirmado

Escolhemos por dois motivos:

O primeiro é que a lista de ingressos de um usuário raramente vai ter mais de 20 itens, então o $O(n^2)$ na prática não faz diferença nenhuma e a implementação é muito mais simples e didática.

O segundo é que o Insertion Sort tem um comportamento muito natural com listas quase ordenadas — se o usuário já tem os ingressos quase em ordem, ele é praticamente $O(n)$, o que é melhor até que o Merge Sort nesses casos.



shared > estruturas > insertion-sort.ts
pages > perfil > historico > historico.ts

Insertion Sort

em dropdowns de filtros

- **Baixo volume de dados**

 Gênero ▾

Gênero

Brega

K-Pop

Pop

Rock

 Localização ▾

Localização

DF

RJ

SP

```
generos = computed(() => { //
  const generosSet = new Set<string>(); //
  this.shows.forEach(show => { //
    if (show.genero) generosSet.add(show.genero); //
  }); //

  const listaGeneros = Array.from(generosSet); //
  return insertionSort(listaGeneros, (a, b) => a.localeCompare(b));
});
```


Quiksort e Lista

Feed

+ Novo Post

Jo

João Silva

21-06-2026 15:43

Essa foi uma das melhores apresentações que já vi! **Energia incrível** do começo ao fim.

87 

Fe

Fernanda Lima

21-06-2026 09:43

Não acredito que finalmente tocaram **Forever Together** ao vivo. Esperei anos por isso!

95 

Lu

Lucas Rocha

21-06-2026 03:43

A interação da banda com o público foi **sensacional**. Dá para sentir o carinho pelos fãs.

203 

Escolhemos por motivos:

Os comentários foram armazenados em uma lista por permitir manipulação simples e eficiente de grandes quantidades de registros.

- Percorrer todos os comentários
- Ordenar
- Filtrar
- Exibir em tela

Para a ordenação foi utilizado QuickSort, devido à sua complexidade média $O(n \log n)$, adequada para cenários com dezenas de milhares de comentários, como o volume estimado de 50.000 registros do sistema.

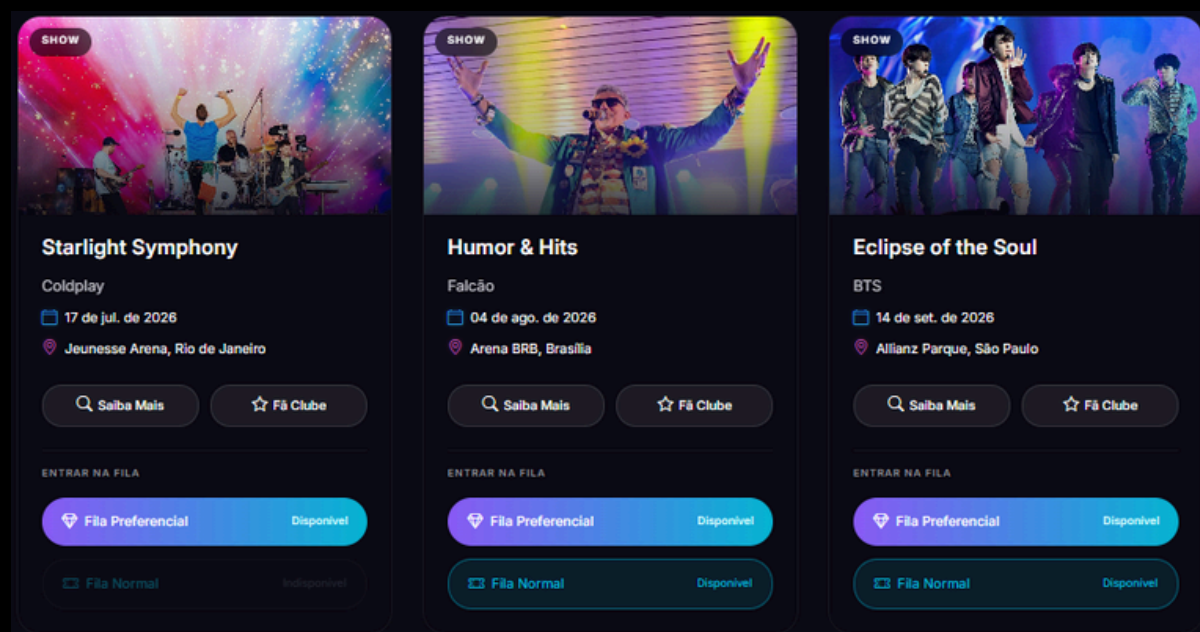
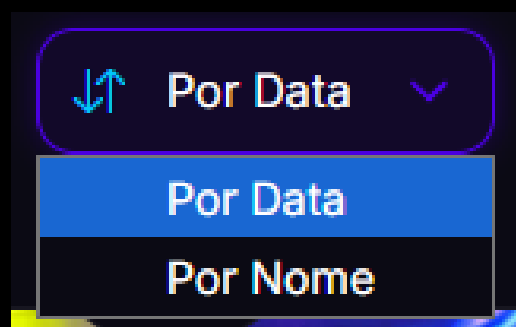
```
TS fila.ts  TS quick-sort.ts X
ingressae > src > app > shared > estruturas > TS quick-sort.ts > QuickSort
1  export class QuickSort {
2      static sort<T>(array: T[], compare: (a: T, b: T) => number): T[] {
3          const copia = [...array];
4
5          this.quickSort(copia, 0, copia.length - 1, compare);
6
7          return copia;
8      }
9
10     private static quickSort<T>(
11         array: T[],
12         inicio: number,
13         fim: number,
14         compare: (a: T, b: T) => number,
15     ): void {
16         if (inicio < fim) {
17             const indicePivo = this.particionar(array, inicio, fim, compare);
18
19             this.quickSort(array, inicio, indicePivo - 1, compare);
20             this.quickSort(array, indicePivo + 1, fim, compare);
21         }
22     }
23 }
```

shared > estruturas >
quick-sort.ts

```
23
24     private static particionar<T>(
25         array: T[],
26         inicio: number,
27         fim: number,
28         compare: (a: T, b: T) => number,
29     ): number {
30         const pivo = array[fim];
31
32         let i = inicio - 1;
33
34         for (let j = inicio; j < fim; j++) {
35             if (compare(array[j], pivo) <= 0) {
36                 i++;
37
38                 [array[i], array[j]] = [array[j], array[i]];
39             }
40         }
41
42         [array[i + 1], array[fim]] = [array[fim], array[i + 1]];
43
44         return i + 1;
45     }
46 }
47 }
```

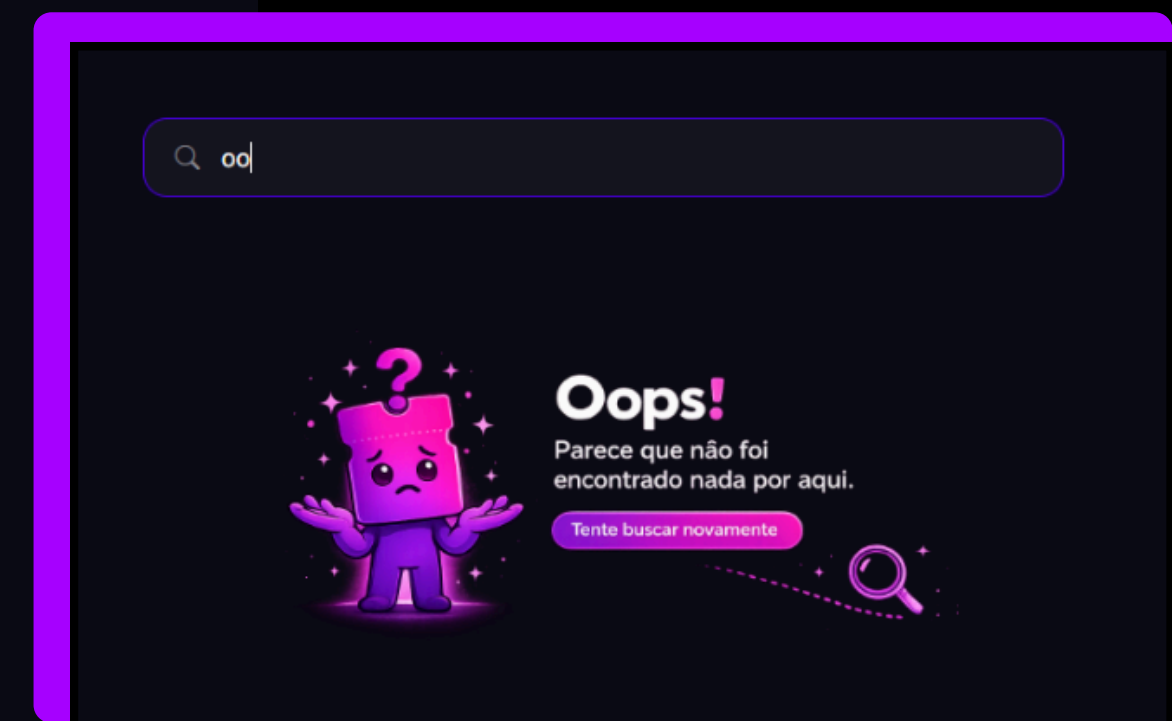
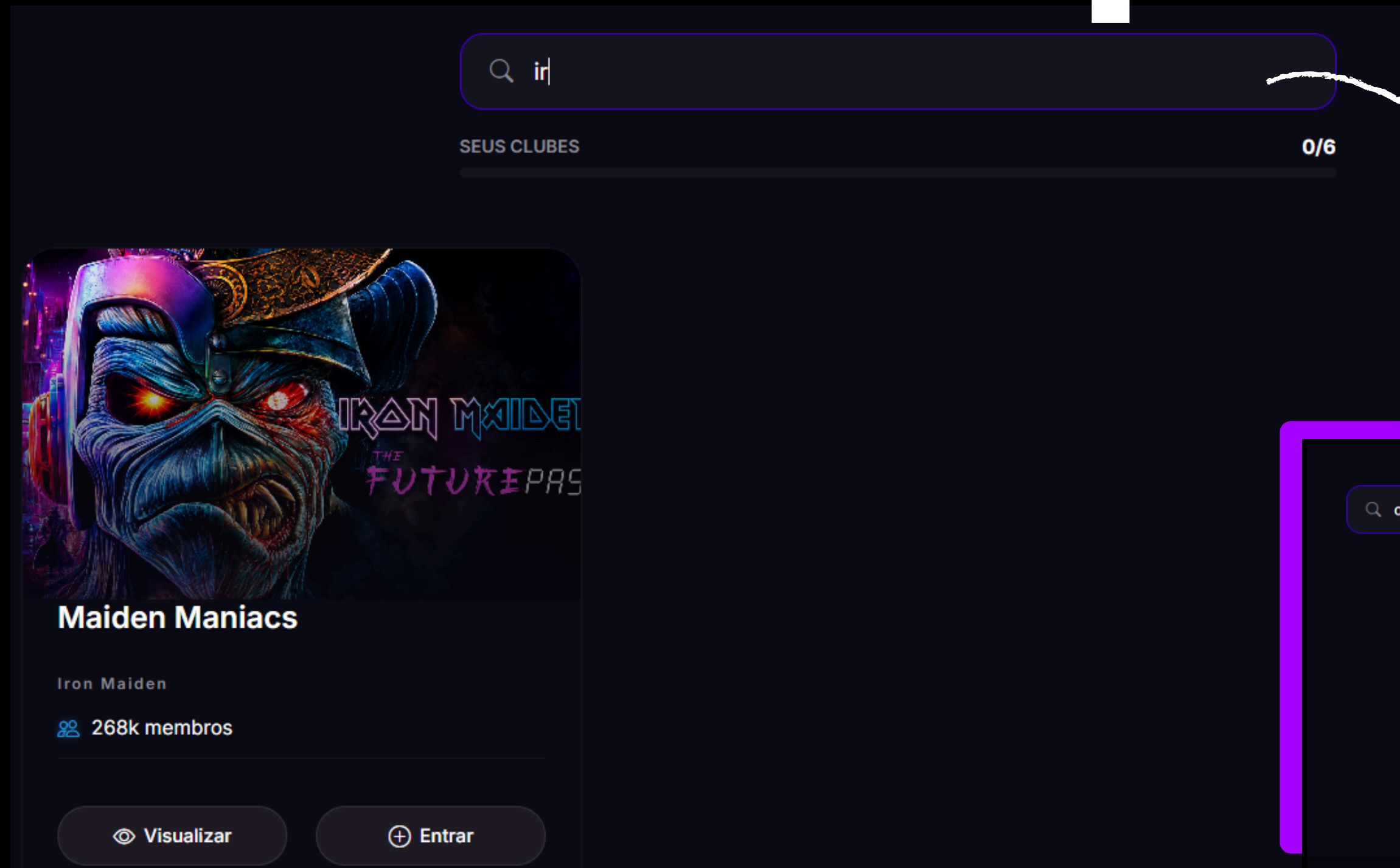

Quick Sort

- Desempenho Geral Altíssimo
- Economia de memória



```
// 5. Ordenação utilizando o algoritmo QuickSort
return QuickSort.sort(resultado, (a, b) => {
  if (filtros.ordenacao === 'nome') { //
    return a.nome.localeCompare(b.nome); //
  } else {
    return new Date(a.dataEvento).getTime() - new Date(b.dataEvento).getTime();
  }
});
```

Busca Sequencial



Escolhemos por três motivos:

- **Volume de Dados Adequado.** Como o número de itens não atinge milhões, pois existe uma alta rotatividade. Assim funciona perfeitamente sem gerar impactos ao usuário.
- Como a busca binária exige que os elementos estejam previamente ordenados segundo o critério de pesquisa, sua utilização não seria adequada nesse cenário.
- **Simplicidade** pois é um algoritmo mais fácil de manter e facilitando futuras modificações

```
ingressae > src > app > shared > estruturas > TS busca-sequencial.ts > BuscaSequencial >
1  export class BuscaSequencial {
2
3      static filtrar<T>(
4          lista: T[],
5          termo: string,
6          campos: (keyof T)[]
7      ): T[] {
8          if (!termo || termo.trim() === '') {
9              return lista;
10         }
11
12         const termoLower = termo.toLowerCase();
13         const resultado: T[] = [];
14
15         // Busca sequencial
16         for (let i = 0; i < lista.length; i++) {
17             for (const campo of campos) {
18                 const valor = String(lista[i][campo]).toLowerCase();
19
20                 if (valor.includes(termoLower)) {
21                     resultado.push(lista[i]);
22                     break;
23                 }
24             }
25         }
26
27         return resultado;
28     }
29 }
```

shared > estruturas >
busca-sequencial.ts

```
fasClubesFiltrados = computed(() => {
    return BuscaSequencial.filtrar(
        this.fasClubes(),
        this.termoBusca(),
        ['nome', 'nomeArtista']
    );
});
```

app > pages > fas-clube > lista.ts

Pilha

Fácil

Médio

Difícil

Hardcore

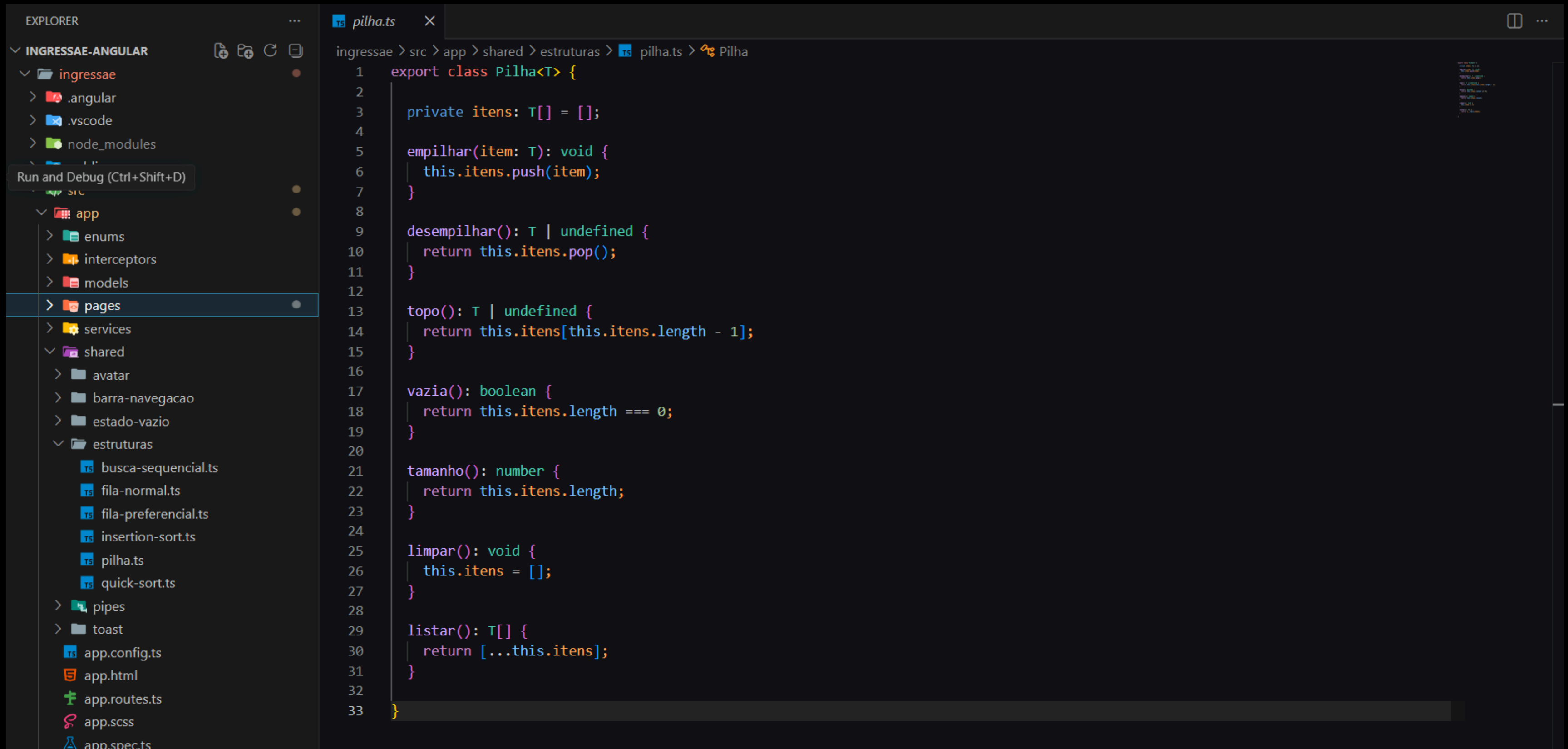
Movimentos: 0

Reiniciar



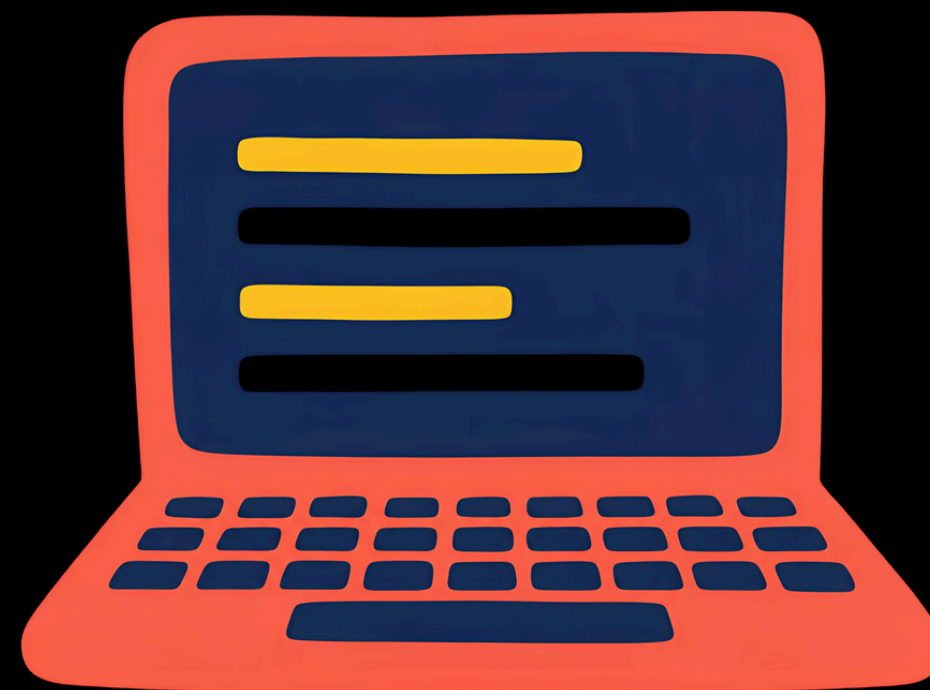
Escolhemos a Torre de Hanoi para demonstrar Pilha por três motivos

- Primeiro, ela representa de forma visual e intuitiva a manipulação de elementos em uma estrutura de dados, facilitando a compreensão das regras e das movimentações realizadas.
- Além disso, o jogo aborda conceitos importantes de algoritmos e estruturas de dados, como organização, sequência lógica de operações e resolução de problemas, agregando um caráter educativo ao sistema.
- Por fim, como a plataforma utiliza uma fila virtual para compra de ingressos, o minigame ajuda a entreter o usuário durante o tempo de espera, tornando a experiência mais agradável e interativa.



shared > estruturas > pilha.ts
pages > fila > jogo > jogo.ts

bora^{pro}



código

Claro que poderíamos ter desenvolvido ainda mais a **área administrativa**, mas optamos por **priorizar as funcionalidades que o usuário realmente irá utilizar**.

Assim, conseguimos entregar uma plataforma mais completa, intuitiva e bem refinada, focando no que gera maior impacto no uso do sistema. A área administrativa pode ser expandida em versões futuras.



obrigado
pela
atenção.